

Construir la app con Spec Kit

AI Media Generator — 39 tickets en 6 fases, listos para ejecutar.

Repo github.com/sjunka/speckit-ai-generator

Feature 001-ai-media-generator

Fecha 2026-08-17

Todo lo que el agente necesita ya está en este repo. Esta guía es para el humano: qué correr, en qué orden, y cómo saber que salió bien.

0. Qué vamos a construir

AI Media Generator. El usuario entra, se registra, toma una foto, elige un estado de ánimo, y recibe una imagen generada por IA. Después convierte esa imagen en un video corto y lo descarga o comparte. Aparte hay un dashboard para un solo dueño: un interruptor que detiene toda la generación, un selector de calidad de video, contadores y un gasto estimado.

Cinco pantallas, cinco rutas de API, unos doce archivos de código. **39 tickets en 6 fases.**

1. Requisitos

Necesitas	Para qué
Claude Code instalado	Ejecuta los comandos <code>/speckit-*</code>
API key de Anthropic funcionando	Sin esto no corre nada
Node 20 o superior	La app es Next.js
git	Ramas y worktrees

No necesitas `uv` ni `specify init`. La maquinaria de Spec Kit ya está commiteada en este repo.

2. Clonar y verificar

```
git clone git@github.com:sjunka/speckit-ai-generator.git
cd speckit-ai-generator
```

Antes de tocar nada, comprueba que Spec Kit encuentra la feature:

```
bash .specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks
```

Debe imprimir algo así:

```
{"FEATURE_DIR":"/.../speckit-ai-generator/specs/001-ai-media-generator","AVAILABLE_DOCS":  
["tasks.md"]}
```

Si dice **Feature directory not found**, falta el puntero. Créalo:

```
printf '{\n  "feature_directory": "specs/001-ai-media-generator"\n}\n' > .specify/feature.json
```

Todos los comandos `/speckit-*` empiezan corriendo ese script. Si falla ahí, no sigas: nada más va a funcionar.

3. Qué hay en el repo

<code>.specify/memory/constitution.md</code>	las 6 reglas que nadie rompe
<code>.specify/feature.json</code>	el puntero a la feature
<code>specs/001-ai-media-generator/</code>	
<code>spec.md</code>	QUÉ hace el producto y POR QUÉ (sin tecnología)
<code>plan.md</code>	CÓMO: stack, contratos, estructura de carpetas
<code>tasks.md</code>	los 39 tickets
<code>docs/</code>	
<code>REPLICATION-PROMPT.md</code>	§6, §7 y §10 son LECTURA OBLIGATORIA
<code>REPLICATION-APPENDIX.md</code>	los 80 archivos fuente, textuales
<code>PROMPT-NOTES.md</code>	por qué el build está partido así
<code>GUIA-SPEC-KIT.md</code>	este archivo

Orden de lectura para un humano

1. `docs/PROMPT-NOTES.md` — el más corto, explica por qué las fases están partidas de esa forma
2. `specs/001-ai-media-generator/spec.md` — qué hace el producto
3. `specs/001-ai-media-generator/plan.md` — los contratos, la parte técnica
4. `specs/001-ai-media-generator/tasks.md` — el trabajo

`REPLICATION-APPENDIX.md` **no se lee**. Es referencia: contiene los 80 archivos fuente exactos y sirve como respuesta correcta cuando dudas de algo.

4. Cómo leer un ticket

En `tasks.md` cada línea se ve así:

```
- [ ] T012 [P] [US1] Construir app/sign-in/[...sign-in]/page.jsx ...
```

Parte	Significa
T012	El número del ticket. Es el que le pasas al comando
[P]	Se puede hacer en paralelo con sus hermanos: toca archivos distintos y no depende de ellos
[US1]	A qué historia de usuario del <code>spec.md</code> pertenece
El texto	Incluye siempre la ruta exacta del archivo

Cuando un ticket termina, el agente cambia `- [ ]` por `- [x]` .

5. Construir, paso a paso

Regla que ahorra dolores

Antes de escribir código, lee `docs/REPLICATION-PROMPT.md` **\$10**. Son 16 trampas, cada una costó una hora la primera vez. El comando `/speckit-implement` carga la constitución, el spec, el plan y los tasks — pero **no** carga los documentos de `docs/` . Ábrelos tú.

Fase 1 — Andamiaje (bloqueante)

```
/speckit-implement T001
```

Crea el proyecto Next.js. Es un solo ticket porque tres agentes no pueden crear el mismo `package.json` .

Verificar: existe `package.json` , `app/layout.jsx` y `app/page.jsx` .

Fase 2 — Fundación (bloqueante)

```
/speckit-implement T002
```

...y sigue hasta T009. Aquí caen los tokens de color, la escala tipográfica, los componentes base, el manifiesto PWA, los dobles de prueba y el CI.

Nadie puede empezar las siguientes fases hasta que esto esté en `main` .

Verificar:

```
npm run lint && npm test && npm run build
```

Los tres en verde, y el CI de GitHub también.

Fases 3, 4 y 5 — en paralelo

Aquí está la gracia del plan. Las tres fases tocan archivos **completamente distintos**, así que tres personas (o tres agentes) trabajan al mismo tiempo sin pisarse.

```
git worktree add ../frontend -b 001-frontend
git worktree add ../backend -b 001-backend
git worktree add ../dashboard -b 001-dashboard
```

Luego, uno en cada carpeta:

Carpeta	Comando	Qué construye
../frontend	/speckit-implement T010 → T017	Landing, sign-in, captura, resultado
../backend	/speckit-implement T018 → T025	Proveedores, almacenamiento, rutas de generación
../dashboard	/speckit-implement T026 → T030	Dashboard, settings, y las cuentas reales

Por qué no chocan: cada fase declara en `tasks.md` qué archivos posee y cuáles nunca toca. Ningún archivo aparece en dos listas. Las llamadas entre fases están fijadas por firma en el `plan.md`, sección *Contracts*, así que cada uno programa contra la firma y mockea lo que todavía no existe.

T030 es especial: es el único ticket que toca cuentas reales (Clerk, MongoDB Atlas, Vercel Blob, Higgsfield). Quien lo tome necesita crear esas cuentas y llenar `.env.local`. Los otros dos siguen trabajando sin credenciales.

Fase 6 — Merge

```
/speckit-implement T031
```

...hasta T039. **T032 es el ticket más importante de esta fase:** reconcilia los mocks de HTTP contra las rutas que de verdad se escribieron. La fase 3 probó contra handlers falsos, no contra el código de la fase 4. Un handler que no coincide con su ruta da tests verdes y app rota.

T038 es la prueba final de exactitud: regenerar el apéndice y compararlo con el original. Diff vacío = terminado.

Si trabajas solo

Ignora los worktrees. Corre las fases en orden numérico, T001 hasta T039. Los bloques de propiedad de archivos no estorban cuando una sola persona los tiene todos, y siguen documentando qué puede tocar cada cambio.

6. Otros comandos útiles

Comando	Cuándo
<code>/speckit-analyze</code>	Antes de empezar. Reporta si spec, plan y tasks se contradicen
<code>/speckit-checklist</code>	Genera listas de verificación de calidad
<code>/speckit-taskstoissues</code>	Convierte los 39 tickets en issues de GitHub, en orden de dependencia

Después de `/speckit-taskstoissues`, `T012` y el issue `#12` son el mismo trabajo, y le pasas al agente el número que te toque.

7. Las 6 reglas que no se rompen

Están completas en `.specify/memory/constitution.md`. Resumidas:

1. **Test primero.** Se commitea la prueba que falla, después el código que la hace pasar. Se verifica en el historial de git.
2. **La suite corre sin internet.** Ninguna prueba necesita API key, red ni base de datos.
3. **Solo se construye lo que está en la lista.** La lista de "fuera de alcance" del spec es obligatoria, no una sugerencia.
4. **Los tokens son el tema.** El color existe una sola vez, en `app/globals.css`. Un hex crudo en cualquier otro lado es un defecto.
5. **La propiedad de archivos define el paralelismo.** Una fase que necesita cambiar un archivo ajeno lo plantea, no lo commitea.
6. **El apéndice manda.** Si la prosa y `REPLICATION-APPENDIX.md` se contradicen, gana el apéndice.

8. Problemas comunes

Síntoma	Causa	Solución
Feature directory not found	Falta <code>.specify/feature.json</code>	Ver paso 2
El comando <code>/speckit-implement</code> no existe	Falta <code>.claude/skills/</code> o usas otro agente	<code>uvx --from git+https://github.com/github/speckit.git specify init --here --integration <tu-agente> --force</code>
El agente regenera <code>tasks.md</code>	Alguien corrió <code>/speckit-tasks</code>	No lo corras. Revierte. El aviso está arriba de <code>tasks.md</code>
Un archivo <code>middleware.js</code> no hace nada	Next 16 lo renombró a <code>proxy.js</code>	Trampa 2 en §10
Una clase de Tailwind no pinta nada	Una clase inventada no da error, simplemente no emite CSS	Trampa 15 en §10
Un botón sale sin estilos	<code>{...props}</code> después de <code>className</code> lo reemplaza todo	Trampa 16 en §10
La imagen generada no se ve	Falta el host en <code>images.remotePatterns</code>	Trampa 13 en §10
El agente inventa detalles	No leyó §6 y §7	Pásale los archivos explícitamente

9. Antes de la presentación

- ☐ Todos pueden clonar el repo
- ☐ `check-prerequisites.sh` imprime un `FEATURE_DIR` en la máquina de cada uno
- ☐ `/speckit-analyze` no reporta contradicciones
- ☐ **Alguien corrió `/speckit-implement T001` de punta a punta y generó el andamiaje**
- ☐ Todos tienen Claude Code y una API key que funciona
- ☐ Todos leyeron `docs/REPLICATION-PROMPT.md` §10

El cuarto punto es el único que no se puede improvisar en vivo. Hazlo antes.

10. Si necesitas cambiar el spec

Los archivos de `specs/` son la fuente de verdad. Edítalos, commitea, y avisa al equipo — los agentes leen la versión que esté en el repo.

Lo único que **no** debes hacer es regenerar `tasks.md` con `/speckit-tasks`. Las tablas de propiedad de archivos son lo que hace seguro el trabajo en paralelo, y una regeneración las pierde. Si necesitas tickets nuevos, escríbelos a mano siguiendo el mismo formato y declara qué archivos posee cada uno.